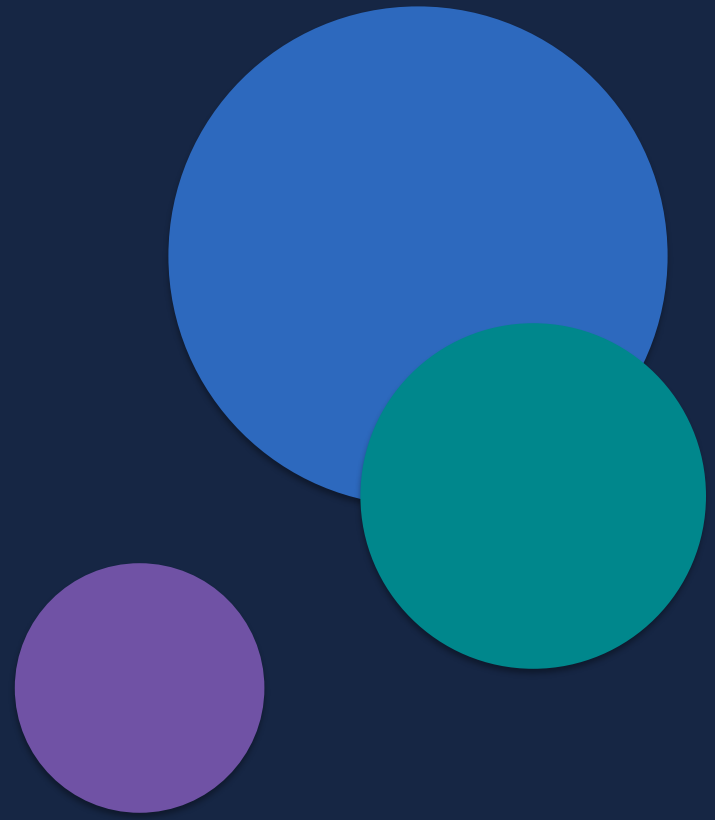
ADVANCED RAG

Complete Concepts, Architecture, Caching & Spring AI

A detailed teaching presentation covering retrieval strategies, query transformation, reranking, Agentic RAG, evaluation, and production-ready caching patterns.



Customer-support knowledge assistant • Instructor-ready deck

How to use this presentation

ROADMAP

- Foundations: what RAG is, where it fits, and how embeddings differ from model weights.
- Indexing: document loading, cleaning, chunking, metadata, embedding models, and vector stores.
- Retrieval: similarity search, top-k, thresholds, metadata filters, hybrid search, and reranking.
- Advanced query patterns: rewriting, multi-query, HyDE, decomposition, parent-document retrieval, and compression.
- Advanced architectures: adaptive, corrective, iterative, Agentic RAG, Graph RAG, and multimodal RAG.
- Caching: embedding, query transformation, retrieval, answer, semantic, prompt-prefix, and ingestion caches.
- Production: evaluation, security, observability, freshness, latency, cost, failure handling, and a reference design.

RAG in one sentence

FOUNDATIONS

Retrieval-Augmented Generation searches an external knowledge source and provides relevant evidence to an LLM before it generates an answer.

RETRIEVAL

Find relevant chunks from documents, search indexes, or data sources.

AUGMENTATION

Add retrieved evidence, instructions, and relevant conversation context to the prompt.

GENERATION

Generate a response grounded in the supplied context, ideally with source references.

RAG improves access to knowledge; it does not guarantee correctness. Retrieval and answer quality must be evaluated.

RAG is not model training

FOUNDATIONS

ML TRAINING

Training examples → learning algorithm → parameter/weight updates → trained predictive model.

Example: train a classifier to predict ticket priority.

RAG INDEXING

Documents → chunks → embeddings → vector index. At question time, retrieve relevant text and pass it to the LLM.

The LLM and embedding model normally keep their existing weights.

A trained embedding model contains learned weights, but embedding a new refund document does not normally retrain that model.

When to use RAG—and when not to

FOUNDATIONS

RAG IS A GOOD FIT

- Internal policies and manuals
- Large knowledge bases
- Answers that need evidence or citations
- Knowledge that changes more often than model training
- Search over private approved documents

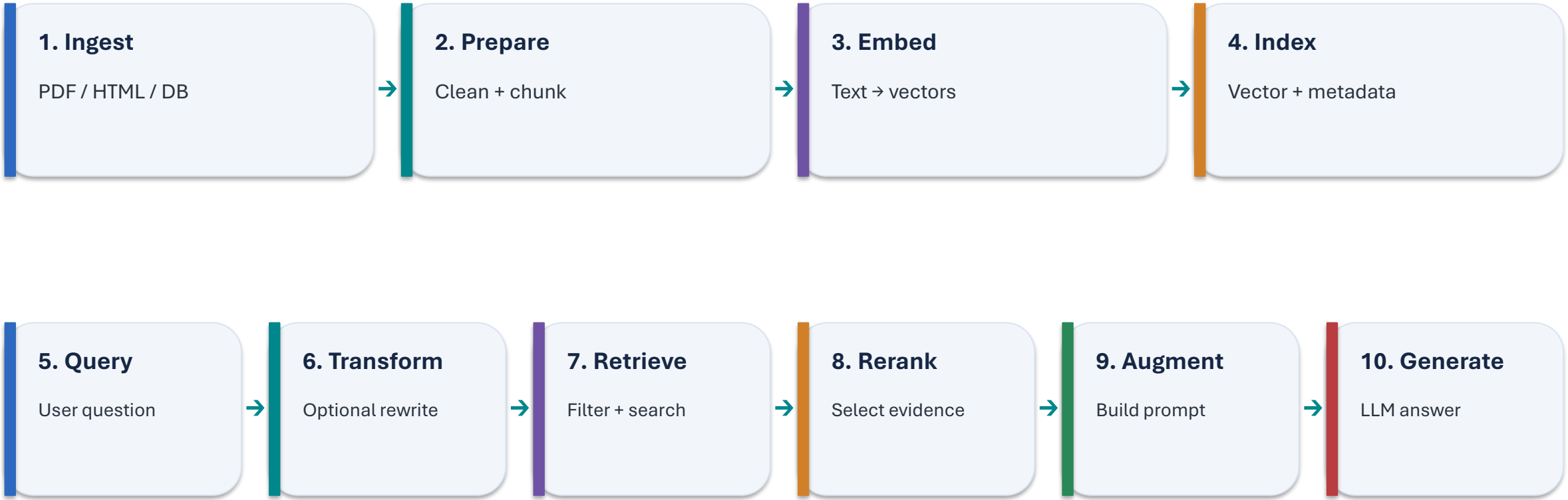
OTHER APPROACH MAY FIT BETTER

- Simple general knowledge: direct LLM
- Exact live account state: API/database tool
- Arithmetic/business rules: deterministic code
- Behavioral adaptation: possibly fine-tuning
- Actions such as issuing a refund: authorized tools/workflows

Many enterprise assistants combine RAG + tools + deterministic business logic rather than choosing only one.

End-to-end RAG architecture

FOUNDATIONS



Caching can sit at multiple stages—but each cache needs a correct key, freshness policy, and access-control boundary.

Ingestion: build the knowledge base

INDEXING

- Load sources: PDF, DOCX, HTML, Markdown, database rows, tickets, and approved APIs.
- Normalize text: remove navigation boilerplate, duplicate headers, malformed characters, and irrelevant markup.
- Preserve structure: headings, page numbers, tables, code blocks, document IDs, source URLs, and effective dates.
- Attach metadata: product, department, tenant, language, version, access policy, source type, and last-updated time.
- Deduplicate and track source versions so updates can replace or delete stale chunks.
- Run ingestion as a repeatable pipeline with metrics, error queues, and reprocessing support.

Chunking strategies

INDEXING

FIXED-SIZE

Split by character/token count, often with overlap. Simple, predictable, but may split a concept mid-sentence.

STRUCTURE-AWARE

Split by headings, paragraphs, HTML sections, tables, or code boundaries. Often a strong default for manuals.

SEMANTIC / HIERARCHICAL

Split by meaning or parent-child structure. Can improve coherence but is more complex and may cost more.

Tune chunk size and overlap using real questions. Small chunks can lose context; large chunks can dilute relevance and consume the context window.

Keep chunk IDs and parent IDs so retrieved evidence can be traced to the original document.

Embeddings and semantic similarity

INDEXING

DOCUMENT EMBEDDING

A trained embedding model maps each chunk to a numeric vector.

QUERY EMBEDDING

The same compatible embedding space represents the user's query as a vector.

SIMILARITY

Search ranks vectors by a metric such as cosine similarity, dot product, or distance.

Use compatible document and query embedding models. Similarity indicates closeness in a learned space—not factual truth or user authorization.

Vector stores: options and selection

INDEXING

SimpleVectorStore	**Demo / unit tests**	In-memory; limited persistence and production features
PostgreSQL + pgvector	**SQL-centric applications**	Relational data and vector search in one platform
Chroma / Qdrant / Milvus	**Self-hosted vector search**	Control over deployment and operational responsibilities
Pinecone / managed services	**Managed cloud**	Less infrastructure management; consider cost and vendor fit
Elasticsearch / Redis / MongoDB Atlas	**Existing data/search platform**	Use vector search alongside platform capabilities

Evaluate persistence, filtering, hybrid search, scale, latency, operations, access control, and total cost.

Basic retrieval: top-k, threshold, filters

RETRIEVAL

TOP-K

Return up to k candidate chunks. Increasing k may improve recall but can add noise and prompt tokens.

SCORE THRESHOLD

Optionally reject weak matches. Calibrate thresholds per model, dataset, and similarity metric.

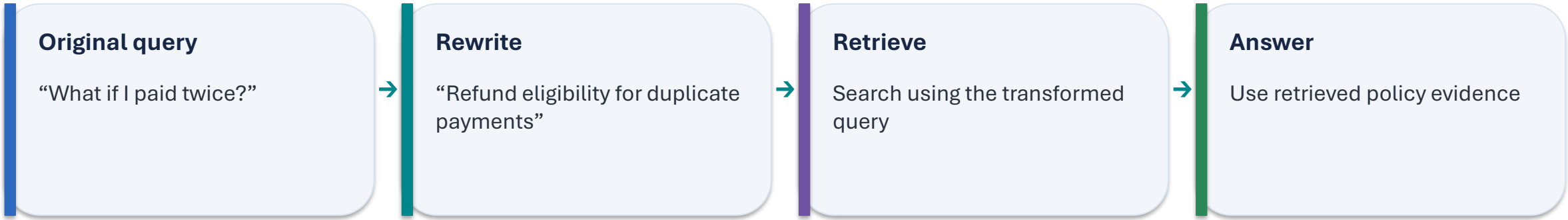
METADATA FILTER

Limit search by product, effective date, tenant, department, or access permissions.

Top-k and similarity thresholds are retrieval controls—not LLM sampling settings like temperature or top-p.

Query transformation / rewriting

ADVANCED RETRIEVAL



- Use when questions are vague, conversational, or refer to previous turns.
- Preserve exact IDs, error codes, product names, dates, negation, and constraints.
- Log original and rewritten queries to debug failures.
- Rewriting is a search aid; it must not silently change the user's intent.

Multi-query retrieval

ADVANCED RETRIEVAL

1. GENERATE VARIANTS

“duplicate payment refund”

“double charge reversal”

“refund for repeated transaction”

2. SEARCH EACH

Run retrieval for each query and collect candidate chunks.

3. MERGE + DEDUPE

Combine results, remove duplicates, and optionally rerank.

Benefit: higher recall across different phrasings. Cost: extra LLM calls/searches, more duplicates, and higher latency.

HyDE: Hypothetical Document Embeddings

ADVANCED RETRIEVAL

- Step 1: ask an LLM to draft a hypothetical passage that might answer the question.
- Step 2: embed that hypothetical passage and search for similar real documents.
- Step 3: use the actual retrieved source documents—not the hypothetical passage—as evidence for the final answer.
- Potential benefit: the generated passage may be closer in style to the documents than the short user query.
- Risk: the hypothetical text can contain incorrect details, so it must never be treated as verified knowledge.

Use when query wording and document wording differ substantially; measure whether it improves retrieval.

Hybrid search: semantic + lexical

ADVANCED RETRIEVAL

DENSE / VECTOR SEARCH

Finds semantic relationships.

“double charged” may match “duplicate payment.”

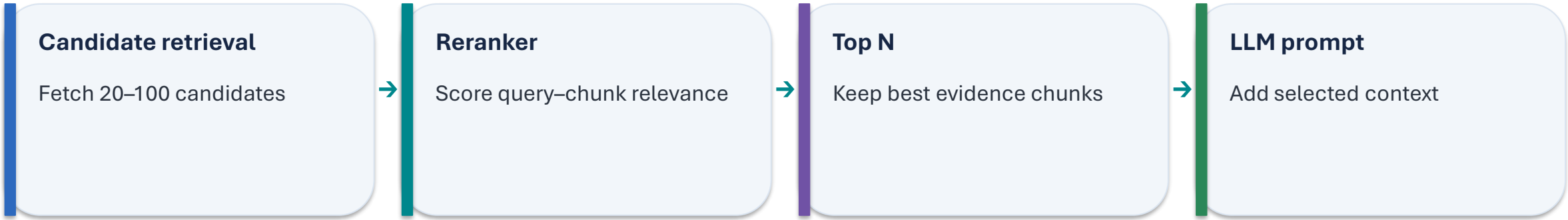
SPARSE / KEYWORD SEARCH

Finds exact tokens, rare terms, identifiers, and codes. BM25 is a common lexical method.

Hybrid systems merge/rank the lists, for example with Reciprocal Rank Fusion (RRF), or use calibrated score fusion.

Reranking: improve the final evidence set

ADVANCED RETRIEVAL



- Embedding search is efficient for broad candidate retrieval.
- A reranker can evaluate the query and each candidate more directly.
- Reranking adds compute and latency, so retrieve a manageable candidate set.
- Measure whether reranking improves recall@k, precision, answer quality, and cost.

Parent-document retrieval

ADVANCED RETRIEVAL

PARENT

A full section, page, or document provides broader context.

CHILD CHUNKS

Small chunks are embedded for precise matching and point to a parent ID.

RETRIEVE + EXPAND

Search child chunks, then return the matching parent section or a bounded window.

Useful when precise matching needs small chunks but correct interpretation requires surrounding policy conditions or explanations.

Metadata filtering and retrieval security

ADVANCED RETRIEVAL

METADATA FILTERING

Filter by active version, product, region, language, policy type, or effective date.

Example: status='active' AND category='refund'.

AUTHORIZATION FILTERING

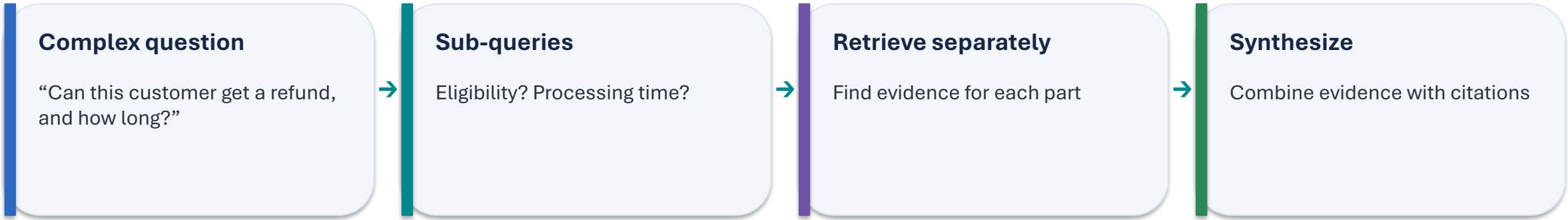
Apply tenant/user permissions in the retrieval layer before context reaches the LLM.

Never rely on the model to enforce access rights.

Test cross-tenant leakage, revoked permissions, stale indexes, prompt injection in source documents, and citations that expose restricted data.

Query decomposition for complex questions

ADVANCED RETRIEVAL



Useful for multi-part and multi-hop questions. Avoid decomposing simple questions unnecessarily: it adds latency and opportunities for errors.

Contextual compression

ADVANCED RETRIEVAL

- Retrieve candidate documents, then keep only the passages relevant to the current question.
- Compression can be extractive (select original sentences) or abstractive (summarize content).
- Preserve dates, conditions, exceptions, qualifiers, numeric values, and source attribution.
- Use compression when retrieved chunks are long, the context window is limited, or irrelevant content distracts the model.
- Retain source IDs and offsets so compressed evidence can be audited against the original.

Adaptive RAG and Corrective RAG

ADVANCED ARCHITECTURES

ADAPTIVE RAG

Choose a path based on query complexity:

- Direct response for simple questions
- Standard retrieval for knowledge questions
- Decomposition or tools for complex questions

Goal: use the right amount of work.

CORRECTIVE RAG

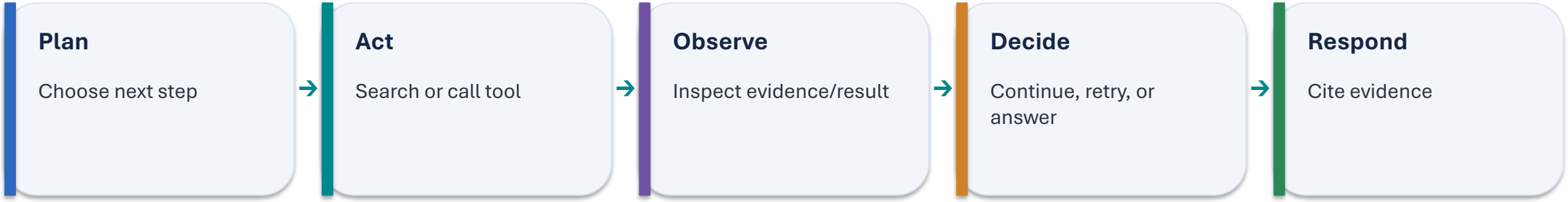
Assess retrieved evidence:

- Relevant and sufficient? Generate answer.
- Weak evidence? Rewrite, broaden, or search another approved source.
- Still unsupported? Say so or ask a clarifying question.

Quality gates can use rules, reranker scores, classifiers, or LLM judges; evaluate false positives and false negatives.

Agentic RAG

ADVANCED ARCHITECTURES



- Example: search refund policy → retrieve order via authorized API → check status → explain eligibility.
- Give tools narrow permissions, validate arguments, set timeouts, and require approval for consequential actions.
- Agentic RAG can solve multi-step tasks but adds latency, cost, and failure modes.
- Do not confuse retrieved knowledge with authority to execute a refund or change an account.

Graph RAG and multimodal RAG

ADVANCED ARCHITECTURES

GRAPH RAG

Uses entities and relationships—often a knowledge graph—to answer questions connecting facts across documents.

Example: customer → order → product → applicable refund rule.

MULTIMODAL RAG

Retrieves or reasons over text plus images, tables, diagrams, audio, or video frames, depending on the models and index design.

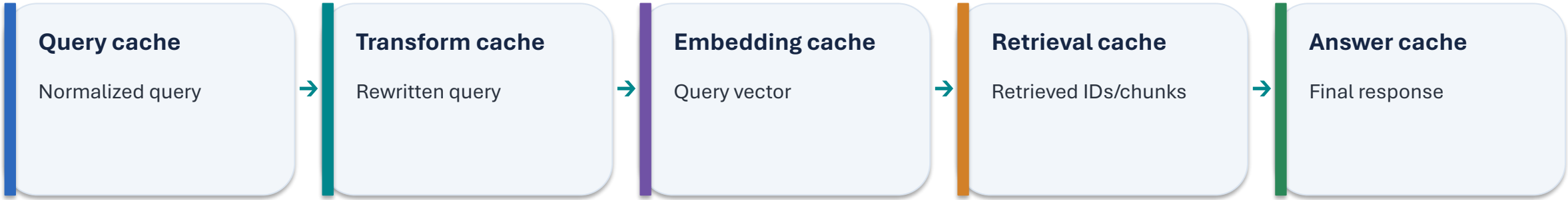
Example: retrieve a screenshot of an error and its troubleshooting steps.

Use Graph RAG for relationship-heavy questions; multimodal RAG when essential evidence is not plain text.

Caching in RAG: why and where

CACHING

Caching stores reusable intermediate results so repeated work can be skipped—reducing latency, model calls, and cost.



- Cache only where the result is safe to reuse and remains valid.
- Each cache needs a key, TTL/freshness rule, invalidation plan, metrics, and access-control strategy.
- Cache hit rate alone is not enough: measure stale answers, correctness, latency, and cost saved.

Cache type 1: embedding cache

CACHING

WHAT TO CACHE

The embedding vector for an identical text chunk or query, so the embedding model need not compute it again.

CACHE KEY

Hash(normalized text + embedding model/version + dimensions + preprocessing version).

Changing the embedding model or preprocessing must create a different key.

Best for repeated text and re-indexing. Keep document and query embeddings in compatible spaces. Avoid leaking sensitive text through keys or cache inspection.

Cache type 2: retrieval-result cache

CACHING

WHAT TO CACHE

The IDs of retrieved chunks, scores, or bounded result payloads for a query and retrieval configuration.

CACHE KEY SHOULD INCLUDE

Normalized/transformed query; tenant and permission scope; filters; vector index/version; embedding model; top-k; reranker/version.

Invalidate or version entries when documents, ACLs, metadata, or indexes change. Re-check authorization before using cached results.

Cache type 3: exact answer cache

CACHING

WHAT TO CACHE

A completed answer for an identical question and equivalent context/configuration.

KEY + GUARDRAILS

Include normalized query, tenant/user scope, context/version, system prompt version, model, locale, and relevant generation settings.

Avoid caching personalized, permission-dependent, live-status, or high-impact answers unless the key and freshness guarantees are rigorous. Do not reuse an answer after policy updates.

Cache type 4: semantic response cache

CACHING

HOW IT WORKS

Embed the new query and look for a semantically similar cached query. If similarity passes a threshold and conditions match, reuse a previous response.

RISKS

Similar wording can hide different intent, user identity, dates, products, or policy context. False cache hits can produce confidently wrong answers.

Use only for low-risk, broadly reusable responses with strict namespace, authorization, version, and similarity controls. Test false-hit rate, not just hit rate.

Other useful caches

CACHING

QUERY-TRANSFORM CACHE

Cache the rewrite for a normalized query plus chat context, rewrite prompt version, and model version. Be careful with conversational references.

RERANK CACHE

Cache reranking scores for the exact query and candidate document IDs/versions. Invalidate when either changes.

PROMPT-PREFIX CACHE

Some model providers cache repeated prompt prefixes internally. This is provider/model-dependent and different from your application's answer cache.

Ingestion idempotency is also valuable: track source hashes and versions to avoid reprocessing unchanged documents.

Reference architecture with caching

CACHING



CACHE MISS

Execute stage, record result, set expiry/version.

CACHE HIT

Validate scope, freshness, and authorization before reuse.

OBSERVE

Track hits, misses, evictions, stale hits, latency, and cost.

TTL, invalidation, and freshness

CACHING

- TTL: expire entries after a defined period. TTL alone may not be sufficient for urgent policy changes.
- Versioning: include document/index/prompt/model versions in keys so old results naturally become unreachable.
- Event invalidation: on document update, delete or invalidate dependent retrieval and answer entries.
- Permission changes: revoke or re-evaluate cached data when access rights change.
- Negative caching: cache “no result” briefly only when it is safe; new documents may appear.
- Stale-while-revalidate: can improve availability for low-risk content, but must not be used blindly for compliance-critical answers.

Caching security and privacy rules

CACHING

NEVER SKIP THESE CHECKS

- Tenant/user namespace separation
- Authorization before serving cached content
- Sensitive-data minimization
- Encryption and access controls for cache storage
- Safe handling of logs and cache keys

BE CAREFUL WITH

- Personalization and conversation history
- Live order/ticket/account status
- Documents with prompt injection
- Shared semantic cache across users
- Cached answers with citations to stale sources

Treat cache contents as data with the same sensitivity as the original retrieved content.

Spring AI: concept-to-component map

SPRING AI

ChatClient	Construct prompt and call/stream the LLM
EmbeddingModel	Create embeddings for documents and queries
VectorStore	Store vectors and retrieve relevant documents
QuestionAnswerAdvisor	Attach retrieved context to the LLM request
QueryTransformer	Rewrite or transform the query before retrieval
SearchRequest	Configure query, top-k, and metadata filters
Document / readers	Represent chunks and load source content
Spring Cache / Redis / Caffeine	Application-level caching options; choose based on persistence and deployment needs

Exact APIs vary by Spring AI version and provider. Verify against the version used by your project.

Illustrative Java cache pattern

SPRING AI

```
// Illustrative pattern: exact cache configuration depends on the app.
@Cacheable(
   cacheNames = "queryEmbeddings",
   key = "#modelVersion + ':' + #normalizedQueryHash"
)
public float[] embedQuery(String normalizedQuery,
                          String modelVersion) {
   return embeddingModel.embed(normalizedQuery);
}

// In production, key construction must also reflect
// preprocessing/model configuration and data boundaries.
// Do not share personalized answers across users by default.
```

This is a design illustration, not a drop-in complete implementation. Add a real cache provider, TTL/eviction policy, metrics, error handling, and tests.

Evaluate retrieval and answer quality

OPERATIONS

RETRIEVAL METRICS

Recall@k, precision@k, MRR, nDCG, relevance judgments, and coverage of expected evidence.

ANSWER METRICS

Correctness, groundedness/faithfulness, citation accuracy, completeness, and appropriate abstention.

SYSTEM METRICS

Latency percentiles, token usage, cost, cache hit rate, stale-hit rate, error rate, and user feedback.

Build a golden dataset of real questions, expected source passages, and acceptable answers. Test changes before rollout.

Observability: trace every RAG decision

OPERATIONS

- Record correlation ID, original query, transformed query, retrieval filters, and index version.
- Capture retrieved document IDs, source versions, scores, reranker output, and selected context size.
- Track cache hit/miss per stage, TTL age, invalidation events, and stale-result detection.
- Record model/provider, prompt version, latency per stage, token counts, and error categories.
- Redact secrets and personal data; logs must obey privacy, retention, and access policies.
- Make it possible to answer: “Why did the system produce this answer, and which sources did it use?”

Common failure modes and remedies

OPERATIONS

Wrong chunks	Improve chunking, hybrid search, metadata filters, query rewrite
Good chunks, wrong order	Add reranking and inspect candidate recall
Answer invents details	Require evidence, citations, abstention, and groundedness checks
Policy changed but old answer appears	Version indexes, invalidate caches, shorten TTL for volatile data
Latency/cost too high	Cache safe stages, reduce candidate count, batch embeddings, tune model choices
Sensitive data leaks	Enforce retrieval ACLs, tenant-scoped keys, and authorization checks on cache hits

Worked example: duplicate payment refund

CASE STUDY

User

“I paid twice. Can I get a refund?”

→

Rewrite

“Eligibility for duplicate-payment refund”

→

Search

Filter active policy; hybrid retrieval

→

Rerank

Rank eligibility and processing-time passages

→

Answer

Cite policy; state conditions

CACHE OPPORTUNITY

Cache query embedding or retrieval results when safe.

LIVE DATA

Use an authorized order/payment API to verify the actual transaction.

GUARDRAIL

Policy explains eligibility; it does not prove this customer paid twice.

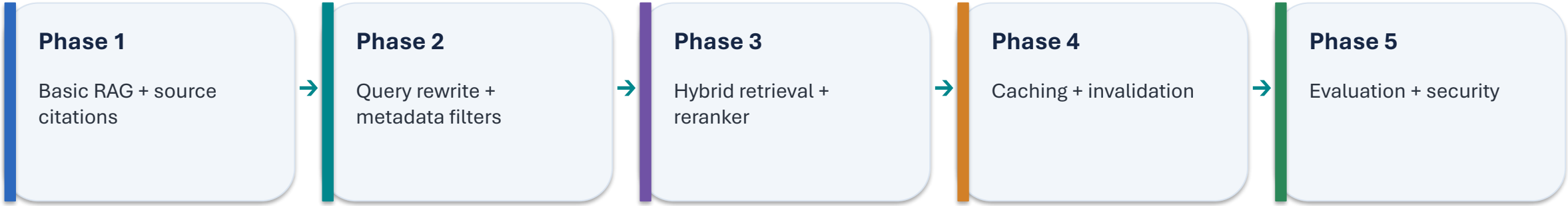
Production design checklist

SUMMARY

- Knowledge: approved sources, document ownership, versioning, freshness, and deletion process.
- Indexing: robust parsing, structure-aware chunks, metadata, compatible embeddings, and deduplication.
- Retrieval: top-k tuning, metadata/ACL filters, hybrid search, reranking, and query transformation where useful.
- Generation: clear system instructions, bounded context, citations, uncertainty handling, and prompt-injection defenses.
- Caching: correct keys, tenant isolation, TTL/versioning, invalidation, metrics, and stale-answer tests.
- Operations: golden test set, tracing, latency/cost budgets, fallbacks, load tests, and incident procedures.

Recommended implementation order

SUMMARY



- Start with a measurable baseline before adding advanced techniques.
- Add one capability at a time and compare quality, latency, and cost against the baseline.
- Use Agentic RAG and Graph RAG only when the problem actually needs multi-step decisions or relationship traversal.

Glossary of key terms

REFERENCE

Chunk	A segment of a source document.
Embedding	A numeric vector generated from text or other content.
Vector store	System that stores vectors and retrieves similar items.
Top-k	Maximum number of candidates requested from a retrieval step.
Reranker	Model that reorders candidates by query relevance.
Hybrid search	Combination of vector/semantic and lexical search.
Query transformation	Rewrite, expand, or decompose a query before retrieval.
Groundedness	How well an answer is supported by the supplied evidence.
Cache invalidation	Rules that prevent stale or unauthorized cached results from being reused.

Key takeaways

CLOSING

- RAG retrieves knowledge; the LLM generates a response from retrieved context.
- Advanced retrieval techniques solve different problems: query rewrite improves the query, hybrid search broadens matching, and reranking improves ordering.
- Agentic and Graph RAG are architectural choices, not mandatory upgrades.
- Caching can reduce latency and cost at several stages, but only with careful keys, access control, freshness, and invalidation.
- Evaluate the complete system—retrieval, answer quality, security, latency, and cost—before calling it production-ready.

End of presentation • Use the worked example as a practical lab.